constrain()

While the `map()` function is useful for changing the start and end values of a variable, it does not limit them to only those values. Leaving values outside of the `map()` function might be handy if we only want to map a small part of a range of values, but leave the rest of the possible values intact. If, though, we wanted to force the values to a very specific, limited range instead, we could use the `constrain()` function. The syntax for this function follows.

```
constrain(value, min, max)
```

The first value is the value that we want to constrain to a specified range. This can be specified as any data type and is not only limited to whole-number integers. The next two values represent the minimum and maximum values in the range.

So hypothetically speaking, if after analyzing our data, looking at the ranges returned by our sensor we find the minimum reported values average around 10 and the maximum tops out at around 900, we could constrain these values before remapping them to a new range. Take the following example code:

```
analogValue = constrain(analogValue, 10, 900);
analogValue = map(analogValue, 10, 900, 0, 255);
```

This code would limit the readings to anything between 0 and 900, ignoring any stray readings that exceed this minimum or maximum, followed by remapping `analogValue` to a more useable range of 0 to 255. Likewise, this same code could be used to, say, ignore readings from a proximity sensor that detects something that is far away, or a light reading beyond a level normally experienced.

Summary

This wraps up the basics behind some of the fun stuff that can be done in an analog world that consists of more than just on and off. You should have a pretty good grasp on how to read an analog sensor, and use these readings to change the brightness of an LED or adjust the speed of a motor. We also talked about how to see these numbers and once you know what they are, how to manipulate them into forms that are more useable for one purpose or another. For a little more information on this subject, you might also check out some of the example analog sketches included with the Arduino programming environment.

For now we are going to move on to a more advanced discussion of functions that includes some functions we haven't discussed before; writing and using custom functions; and a special kind of function that will allow us to stop what we are doing in a sketch and do something else all together. These can be pretty useful as we develop a deeper understanding of writing code. We will, of course, continue to return to both digital and analog functions throughout the rest of the book because they are not only fundamental to what we are doing, but entirely essential.